Neurolabs iOS SDK Integration Guide

Technical Integration Specification

Document ID: NLB-IOS-INTEGRATION-GUIDE
Version: v1.2.5
Date: 2026-04-24
Status: Partner integration reference
Owner: Neurolabs

1. Scope

This guide is for partner iOS apps integrating ‘NeurolabsSDK‘ through Swift Package Manager.

2. Changes In v1.2.3

- Default task routing config uses ‘taskUUID‘.
- Recommended custom-camera parity flow remains strict shelf guidance with overlays disabled.
- ‘liveQualityEnabled: true‘ remains the key switch for pill/rotation/warning/error guidance behavior.
- ‘autoCloseAfterCapture: true‘ with a single required capture remains the recommended parity flow.
- ‘NLCameraConfiguration.captureRange(...)‘ is the preferred helper for custom-camera flows that need a minimum capture count and a higher maximum cap.
- ‘showsCapturedCountLabel‘ hides the ‘N captured‘ chip in custom camera and multi-capture flow UIs.

3. Requirements

- iOS 17.0+ (SDK package declares ‘.iOS(.v17)‘)
- Xcode 16.4+ (release toolchain baseline)
- Swift 5.9+ (Swift 6 toolchains supported)

4. Install (SPM)

Add package dependency (mobile-dist or direct distribution manifest) and link ‘NeurolabsSDK‘.

5. SDK Initialization + Warmup

```
import NeurolabsSDK

final class CaptureCoordinator {
    let sdk = NeurolabsSDKCore(configuration: .init(
        apiKey: "<API_KEY>",
        apiBaseURL: URL(string: "https://api.neurolabs.ai")!,
        taskUUID: "<DEFAULT_TASK_UUID>",
        debugLogging: true
    ))

    func prepare() {
        // SDK starts image pipeline preparation during init.
        // Optional: observe loading progress through delegate.
    }
}
```

6. Queue Management

```
Task {
    await sdk.setAutoSyncEnabled(true)
    await sdk.setWifiOnlyUploadsEnabled(false)
    await sdk.setDeletePhotosOnUploadSuccess(true)
    await sdk.setMaxQueueSize(200)
    await sdk.setUploadRetryCount(5)

    let status = await sdk.getQueueStatus()
    print("pending=\(status.pendingCount) failed=\(status.failedCount)")

    await sdk.flushQueue()
    await sdk.retryFailedUploads()
}
```

7. Open Custom Camera

```
import UIKit
import NeurolabsSDK

var capture = NLCaptureConfiguration()
capture.confidenceThreshold = 0.25
capture.iouThreshold = 0.45
capture.maxCaptures = 1
capture.guidanceMode = .strict
capture.showDetectionOverlays = false
capture.showCapturedRegions = false

let config = NLCameraConfiguration.captureRange(
    capture,
    minimumCaptures: 1,
    maximumCaptures: 5,
    cameraMode: .default,
    showCapturePreview: true,
    showPreviewInStrictMode: true,
    showsCapturedCountLabel: false,
    enableValidation: true,
    showAlignmentGuidance: true,
    enableARDetections: false,
    liveQualityEnabled: true,
    liveQualityFPS: 6
)

let handlers = NLCustomCameraHandlers(
    onSave: { capture in
        // Custom post-processing hook for each accepted capture
        // return .performDefault -> SDK also enqueues upload
        // return .skipDefault -> app takes full ownership
        return .performDefault
    },
    onSaveAll: { captures in
        // Batch hook (multi-capture flow)
        return .performDefault
    }
)

sdk.openCustomCameraUI(
    configuration: config,
    handlers: handlers,
    onClose: {
        print("Camera closed")
    }
)
```

Recommended capture payload notes:

- ‘liveQualityEnabled: true‘ is the key for pill/rotation/warning/error guidance behavior in the custom camera.
- Keep shelf capture mode with strict guidance and overlays disabled for the custom-camera guidance UI path.
- Use ‘NLCameraConfiguration.captureRange(...)‘ to express a minimum capture requirement plus a larger maximum cap.
- Use ‘showsCapturedCountLabel = false‘ to hide the count chip when you do not want ‘N captured‘ shown.

- ‘guidanceMode: .guidance‘ should only be used as a temporary fallback if a partner explicitly wants looser guidance than the parity path.

8. WebView Bridge Example

The same capture-range options are available through the native bridge models:

```
let config = NLNativeCaptureConfig(
    guidanceMode: .strict,
    showCloseButton: false,
    maxCaptures: 5,
    allowManualFinish: true,
    minCapturesBeforeDone: 1,
    showsCapturedCountLabel: false,
    cameraMode: .default
)

neurolabsSDK.openNativeCaptureUI(config: config, sessionId: sessionId)
```

Use ‘NLNativeCaptureOptions‘ with the same field names when the bridge payload is built from JS or other app code.

9. Post-Processing Hooks

- ‘onSave‘ receives each approved capture.
- ‘onSaveAll‘ receives final batch in multi-capture mode.
- ‘onUploadRequested‘ exists but default queueing is done in ‘onSave‘ / ‘onSaveAll‘ path.

```
let handlers = NLCustomCameraHandlers(
    onSave: { capture in
        MyPostProcessor.shared.enqueue(capture)
        return .skipDefault
    }
)
```

9. Delegate/Event Callbacks

```
final class SDKDelegate: NeurolabsSDKDelegate {
    func neurolabsSDK(_ sdk: NeurolabsSDKCore, didProduceCaptureResult result: NLNativeCaptur
eResult, for captureId: String, sessionId: String) {}
    func neurolabsSDK(_ sdk: NeurolabsSDKCore, didEncounterError error: NLError, sessionId: S
tring?, messageId: String?) {}
    func neurolabsSDK(_ sdk: NeurolabsSDKCore, didChangeQueueStatus status: NLQueueStatus) {}
    func neurolabsSDK(_ sdk: NeurolabsSDKCore, didUpdateLoadingProgress progress: NLLoadingPr
ogress) {}
}

sdk.delegate = SDKDelegate()
```

10. Notes

- Per-session routing is supported via native capture config task UUID overrides.
- Ensure ‘NSCameraUsageDescription‘ is present in app ‘Info.plist‘.